Aga - Conceptual Design & Relationship Modeling
- intro
- new review requirement
- relationship attribute reasoning
- list subtype design reasoning

Tola - Relational Mapping & Constraint Enforcement
- map Aga's design into relations
- schema.sql changes
- keys/FKs/CHECK constraints
- what DDL does and does not enforce

Chris - Improving Conceptual Design Quality
- certified curator attribute
- updated ERD
- completeness / correctness / minimality
- certified-first review retrieval concept

Cooper - Constraints, Derived Data & Verification
- relational algebra constraints

Rates

Let W^+ be the set of whitespace strings of length ≥ 1
Let B (blank strings) = { s ∈ Strings | |s|=0 ∨ s ∈ W+ }
$\sigma_{review \in B}$ (Rates) = ∅

SongLists

Let S be the set of list_id values in SongLists
Let L be the set of list_id values in Lists
Every ID in SongLists must also be in Lists
$\pi_{list_id}(SongLists) \subseteq \pi_{list_id}(Lists)$

AlbumLists

Let S be the set of list_id values in AlbumLists
Let L be the set of list_id values in Lists
Every ID in AlbumLists must also be in Lists
$\pi_{list_id}(AlbumLists) \subseteq \pi_{list_id}(Lists)$

- trending query

Trending Rule:
A user can contribute to the trending count once and only once by one of these actions

1. rating an album
2. adding an album to a list

1. A user rated an album:

WHO rated the album, the (username, album_id) pair from Rates represents a user rating an album once, which counts towards a trending point.

```
SELECT username, album_id FROM Rates;
```

2. A user added an album to a list:
WHO added the album to a list, by finding the list that contains an album and using its list_id to find the list's owner (username) in Lists, the owner (user), along with the album they added to the list gives us a (username, album_id) pair, which counts toward a trending point.

```
FROM ListAlbums JOIN Lists
ON ListAlbums.list_id = Lists.list_id;
```

3. Final table:
Duplicate (username, album_id) pairs can exist across both tables, e.g. a user rated an album and added it to multiple lists, but a union between between all such (username, album_id) pairs returns a table where duplicates are removed, thus enforcing the Trending Rule.

```
SELECT username, album_id FROM Rates

UNION

SELECT Lists.username, ListAlbums.album_id
FROM ListAlbums JOIN Lists
ON ListAlbums.list_id = Lists.list_id;
```

- test suite

- live/demo evidence

- final schema build

Aga - Sprint Integration & Next Steps
- written album reviews
- certified curator metadata
- proper SongList / AlbumList subtype design
- relational constraints for new rules
- derived trending logic
- certified-first review retrieval
- some business rules are still application-level
- subtype total participation may not be fully enforced

- refine constraints and schema where necessary
- update ERD/DDL together
- continue testing new features without duplicating derived data